

Capítulo 5: Redes neuronales

Parte 2: optimización y backpropagation

Basado en C. M. Bishop, *Pattern Recognition and Machine Learning*

Ruta

1. Planteamiento: entrenamiento como optimización de $E(\mathbf{w})$.
2. Aproximación: geometría local con gradiente y Hessiano.
3. Gradientes: descenso por gradiente y SGD.
4. Backpropagation: cálculo eficiente de derivadas.
5. Jacobiano: sensibilidad de la salida respecto a la entrada.

Idea

La sesión conecta tres niveles: geometría de la optimización, regla de la cadena y algoritmo computacional.

Idea clave

Una red feed-forward define un mapa no lineal

$$\mathbf{x} \mapsto \mathbf{y}(\mathbf{x}, \mathbf{w}),$$

y el entrenamiento consiste en elegir $\mathbf{w}$ para minimizar una función de error $E(\mathbf{w})$ adecuada al tipo de tarea.

- ▶ Regresión: salida lineal y error cuadrático.
- ▶ Clasificación binaria: salida sigmoide y entropía cruzada binaria.
- ▶ Clasificación multiclase: salida softmax y entropía cruzada multiclase.

Problema de optimización de una red neuronal

Dada una función de error $E(\mathbf{w})$, ¿cómo encontrar eficientemente parámetros buenos?

Definición

- ▶ $\mathbf{x} \in \mathbb{R}^D$: vector de entrada.
- ▶ t o $\mathbf{t}$: objetivo (target).
- ▶ $\mathbf{y}(\mathbf{x}, \mathbf{w})$: salida de la red.
- ▶ $\mathbf{w} \in \mathbb{R}^W$: vector total de parámetros.
- ▶ $E(\mathbf{w})$: error total.
- ▶ $E_n(\mathbf{w})$: contribución del dato n al error total.

Definición

- ▶ $\nabla E(\mathbf{w})$: gradiente respecto a los pesos.
- ▶ $H = \nabla \nabla E(\mathbf{w})$: Hessiano.
- ▶ a_j : activación pre-no lineal de la unidad j .
- ▶ z_j : activación post-no lineal de la unidad j .
- ▶ $\delta_j = \partial E_n / \partial a_j$: sensibilidad local.
- ▶ $J_{ki} = \partial y_k / \partial x_i$: Jacobiano.

Definición

$$\mathbf{w}^* = \arg \min_{\mathbf{w}} E(\mathbf{w}).$$

- ▶ El problema ya no vive en el espacio de entrada, sino en el espacio de parámetros.
- ▶ Si la red tiene W parámetros, el espacio de búsqueda es $\mathbb{R}^W$.
- ▶ En redes neuronales, la dependencia de E respecto de $\mathbf{w}$ suele ser **no convexa**.
- ▶ $E(\mathbf{w})$ define una superficie de error en el espacio de pesos, es decir, le asigna cierto valor a cada valor posible de $\mathbf{w}$.

Cambio local del error ante un pequeño desplazamiento

Si nos movemos de $\mathbf{w}$ a $\mathbf{w} + \delta\mathbf{w}$, entonces la expansión de Taylor de primer orden da

$$E(\mathbf{w} + \delta\mathbf{w}) = E(\mathbf{w}) + \delta\mathbf{w}^T \nabla E(\mathbf{w}) + O(\|\delta\mathbf{w}\|^2).$$

Por tanto, despreciando términos de orden cuadrático,

$$\delta E = E(\mathbf{w} + \delta\mathbf{w}) - E(\mathbf{w}) \approx \delta\mathbf{w}^T \nabla E(\mathbf{w}).$$

Idea

- ▶ La fórmula es válida para desplazamientos pequeños $\delta\mathbf{w}$.
- ▶ $\nabla E(\mathbf{w})$ apunta en la dirección de crecimiento más rápido de E .
- ▶ Por tanto, $-\nabla E(\mathbf{w})$ apunta en la dirección de descenso local más pronunciado.

Advertencia

Esta es una aproximación local de primer orden; su validez depende de que el desplazamiento sea suficientemente pequeño.

Fuente: Bishop, Sección 5.2.1.

Definición

Un punto $\mathbf{w}$ es **estacionario** si

$$\nabla E(\mathbf{w}) = \mathbf{0}.$$

- ▶ Un punto estacionario puede ser un mínimo local.
- ▶ También puede ser un máximo local.
- ▶ O puede ser un punto de silla.

Advertencia

Que el gradiente se anule es una condición *necesaria* para un mínimo interior, pero no *suficiente*.

Idea

En una red con unidades ocultas, distintos vectores de pesos pueden representar exactamente la *misma* función de entrada-salida.

- ▶ Permutar unidades ocultas no cambia el mapa total.
- ▶ En ciertas activaciones (por ejemplo $\tanh$), también hay simetrías por cambios de signo compensados.
- ▶ Por tanto, una misma solución funcional puede aparecer muchas veces en el espacio de pesos.

Advertencia

La superficie de error no solo tiene múltiples mínimos; algunos corresponden a descripciones paramétricas distintas de la misma función.

1. La función $E(\mathbf{w})$ suele ser **no convexa**.
2. El espacio de búsqueda puede tener dimensión grande.
3. Hay múltiples puntos estacionarios.
4. Existen simetrías y redundancias en la parametrización.
5. La dependencia de la salida respecto de los parámetros es altamente no lineal.

Idea

Necesitamos métodos numéricos iterativos y una forma eficiente de explotar información local sobre la superficie de error.

Antes de pasar a varias variables, recordemos la expansión de Taylor de segundo orden en una variable:

$$f(x) \approx f(x_0) + f'(x_0)(x - x_0) + \frac{1}{2}f''(x_0)(x - x_0)^2.$$

Idea

- ▶ $f(x_0)$ da el valor local.
- ▶ $f'(x_0)$ describe la pendiente local.
- ▶ $f''(x_0)$ describe la curvatura local.

La generalización multivariable hará aparecer al gradiente y al Hessiano.

Alrededor de un punto $\hat{\mathbf{w}}$ en el espacio de pesos,

$$E(\mathbf{w}) \approx E(\hat{\mathbf{w}}) + (\mathbf{w} - \hat{\mathbf{w}})^\top \mathbf{b} + \frac{1}{2}(\mathbf{w} - \hat{\mathbf{w}})^\top H(\mathbf{w} - \hat{\mathbf{w}}),$$

donde

$$\mathbf{b} = \nabla E \Big|_{\mathbf{w}=\hat{\mathbf{w}}}, \quad H_{ij} = \frac{\partial^2 E}{\partial w_i \partial w_j} \Big|_{\mathbf{w}=\hat{\mathbf{w}}}.$$

- ▶ $\mathbf{b}$ es el gradiente local.
- ▶ H recoge la curvatura local.

Fuente: Bishop, Sección 5.2.2.

Derivando la expansión anterior con respecto a $\mathbf{w}$,

$$\nabla E(\mathbf{w}) \approx \mathbf{b} + H(\mathbf{w} - \hat{\mathbf{w}}).$$

Idea

El gradiente no es constante: cambia al movernos en el espacio de pesos, y ese cambio local queda gobernado por el Hessiano.

- ▶ Si H es grande en una dirección, la pendiente cambia rápido al movernos en esa dirección.
- ▶ Si H es pequeño, la superficie es más plana allí.

Si $\mathbf{w}^*$ es un mínimo local, entonces

$$\nabla E(\mathbf{w}^*) = \mathbf{0}.$$

Por tanto, la expansión se simplifica a

$$E(\mathbf{w}) \approx E(\mathbf{w}^*) + \frac{1}{2}(\mathbf{w} - \mathbf{w}^*)^\top H(\mathbf{w} - \mathbf{w}^*).$$

Advertencia

Cerca de un mínimo, la forma local de la superficie de error viene determinada completamente por el Hessiano.

Si H es simétrico, podemos escribir

$$Hu_i = \lambda_i u_i, \quad u_i^\top u_j = \delta_{ij}.$$

- ▶ u_i : dirección principal de curvatura.
- ▶ λ_i : curvatura asociada a esa dirección.

Idea

Autovalores grandes significan curvatura fuerte; eigenvalores pequeños significan curvatura suave o una dirección relativamente plana.

Expandimos el desplazamiento respecto al mínimo en la base de eigenvectores:

$$\mathbf{w} - \mathbf{w}^* = \sum_i \alpha_i \mathbf{u}_i.$$

Al sustituir en la expansión cuadrática obtenemos

$$E(\mathbf{w}) \approx E(\mathbf{w}^*) + \frac{1}{2} \sum_i \lambda_i \alpha_i^2.$$

Idea

En estas coordenadas, la aproximación local se desacopla en sumas de términos cuadráticos independientes.

Fuente: Bishop, Sección 5.2.2.

Se dice que H es **positivo definido** si

$$\mathbf{v}^T H \mathbf{v} > 0 \quad \text{para todo } \mathbf{v} \neq \mathbf{0}.$$

Teorema

En un punto estacionario, si el Hessiano es positivo definido, entonces estamos ante un mínimo local.

- ▶ En una dimensión: basta que $E''(w^*) > 0$.
- ▶ En varias dimensiones: necesitamos positividad en *todas* las direcciones.

Idea

Cerca de un mínimo, **gradiente** y **Hessiano** son la información local dominante.

¿Qué información obtenemos en una evaluación de E ?

Evaluación directa

$$E(\mathbf{w})$$

Una evaluación devuelve **un número**.

Evaluación del gradiente

$$\nabla E(\mathbf{w})$$

Una evaluación devuelve **W números**, uno por parámetro.

El gradiente contiene mucha más información local sobre la forma de la función que una sola evaluación escalar de E .

Definición

$$\mathbf{w}^{(\tau+1)} = \mathbf{w}^{(\tau)} - \eta \nabla E(\mathbf{w}^{(\tau)}),$$

donde $\eta > 0$ es la tasa de aprendizaje.

- ▶ τ indexa la iteración.
- ▶ El nuevo punto se obtiene desplazándose en dirección opuesta al gradiente.

Idea

Si el gradiente apunta hacia donde el error sube más rápido, su opuesto apunta hacia donde desciende localmente con mayor rapidez.

Fuente: Bishop, Sección 5.2.4.

- ▶ **Conjugate gradients:** reutiliza información de iteraciones anteriores para construir direcciones de búsqueda mejores.
- ▶ **Quasi-Newton:** aproxima información de curvatura sin calcular explícitamente todo el Hessiano.

Advertencia

Es común que el primer método que se aprende para entrenar una red neuronal es gradient descent porque es conceptualmente simple, no porque sea siempre la mejor opción práctica.

Fuente: Bishop, Sección 5.2.4.

- ▶ Entrenar una red significa minimizar una función de error no convexa en el espacio de pesos.
- ▶ La aproximación cuadrática local muestra el papel del gradiente y del Hessiano.
- ▶ Usar gradientes es esencial por razones tanto geométricas como computacionales.
- ▶ Ahora necesitamos un procedimiento **eficiente** para calcular esos gradientes.

Definición

Backpropagation es un procedimiento eficiente para calcular derivadas de la función de error con respecto a los pesos de una red feed-forward.

- ▶ No es sinónimo de *arquitectura* de red.
- ▶ No es sinónimo de *entrenamiento completo*.
- ▶ No es sinónimo de *descenso por gradiente*.

Advertencia

Backpropagation resuelve la etapa de **evaluación de derivadas**; luego un método de optimización decide cómo usar esas derivadas para actualizar los pesos.

Fuente: Bishop, inicio de la Sección 5.3.

Cálculo de derivadas

$$\frac{\partial E}{\partial w}$$

Esto lo hace backpropagation.

Actualización de pesos

$$\mathbf{w}^{(\tau+1)} = \mathbf{w}^{(\tau)} + \Delta \mathbf{w}^{(\tau)}$$

Esto lo hace el algoritmo de optimización.

Separar ambas etapas evita la confusión clásica de pensar que backpropagation es el entrenamiento completo.

Cada unidad recibe una combinación lineal de sus entradas:

$$a_j = \sum_i w_{ji} z_i,$$

y luego aplica una no linealidad diferenciable:

$$z_j = h(a_j).$$



- ▶ z_i : activación de una unidad que envía conexión hacia j .
- ▶ w_{ji} : peso asociado a la conexión $i \rightarrow j$.
- ▶ Un sesgo se maneja agregando una entrada fija $z_0 = 1$.

Fuente: Bishop, Sección 5.3.1.

En algunos métodos de optimización, el error total se descompone como

$$E(\mathbf{w}) = \sum_{n=1}^N E_n(\mathbf{w}).$$

Para una red general, el peso w_{ji} entra primero en la suma ponderada de la unidad j :

$$a_j = \sum_r w_{jr} z_r.$$

Por eso, si tomamos a_j como variable intermedia inmediata, la regla de la cadena da

$$\frac{\partial E_n}{\partial w_{ji}} = \frac{\partial E_n}{\partial a_j} \frac{\partial a_j}{\partial w_{ji}}.$$

Idea

La dependencia relevante puede verse como

$$w_{ji} \longrightarrow a_j \longrightarrow z_j \longrightarrow \cdots \longrightarrow E_n.$$

Al derivar respecto a w_{ji} , el primer cambio que produce ese peso ocurre en a_j .

Introducimos la notación

$$\delta_j \equiv \frac{\partial E_n}{\partial a_j}.$$

Definición

Mide cuánto cambiaría el error del dato n si perturbáramos ligeramente la activación pre-no lineal a_j de la unidad j .

- ▶ A veces se le llama “error” de la unidad.
- ▶ Más precisamente, es una **sensibilidad local**.

Como

$$a_j = \sum_i w_{ji} z_i,$$

se tiene

$$\frac{\partial a_j}{\partial w_{ji}} = z_i.$$

Por tanto,

$$\frac{\partial E_n}{\partial w_{ji}} = \delta_j z_i.$$

Idea

La derivada respecto a un peso es el producto entre:

- ▶ la sensibilidad local en el extremo de salida de la conexión;
- ▶ la activación que entra por el extremo de entrada.

Fuente: Bishop, Ecuaciones (5.50)–(5.53).

Caso especial: conexiones desde la entrada

La fórmula general

$$\frac{\partial E_n}{\partial w_{ji}} = \delta_j z_i$$

usa z_i para denotar la **activación de la unidad que alimenta a j** .

En una capa oculta que recibe directamente la entrada, esas “activaciones anteriores” son simplemente las componentes del dato:

$$z_i \equiv x_i, \quad i = 0, 1, \dots, D,$$

con $x_0 = 1$ para representar el sesgo.

Conclusión

Para los pesos que conectan la entrada con la capa oculta,

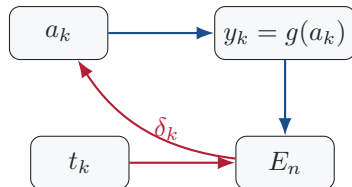
$$\frac{\partial E_n}{\partial w_{ji}^{(1)}} = \delta_j^{(1)} x_i.$$

No es una excepción: es la misma regla $\delta_j z_i$, usando $z_i = x_i$ en la capa de entrada.

Para los emparejamientos naturales entre activación de salida y pérdida,

$$\delta_k = \frac{\partial E_n}{\partial a_k} = y_k - t_k.$$

- ▶ Regresión: salida lineal y SSE.
- ▶ Binaria: sigmoide y entropía cruzada.
- ▶ Multiclase: softmax y entropía cruzada.



Notación

δ_k es la sensibilidad del error del dato n respecto a la preactivación de salida a_k .

¿Cómo obtenemos δ_j en una unidad oculta?

Una unidad oculta no aparece directamente en la pérdida. Su efecto llega al error a través de las unidades posteriores a las que alimenta.

Si la unidad oculta j envía conexiones a varias unidades k , entonces el cambio en a_j puede afectar a todos esos a_k . Por la regla de la cadena multivariable,

$$\delta_j = \frac{\partial E_n}{\partial a_j} = \sum_k \frac{\partial E_n}{\partial a_k} \frac{\partial a_k}{\partial a_j}.$$

Idea

La suma aparece porque hay varios caminos desde a_j hasta la pérdida: uno por cada unidad posterior k conectada con j .

Fórmula recursiva de backpropagation

Partimos de

$$\delta_j = \sum_k \frac{\partial E_n}{\partial a_k} \frac{\partial a_k}{\partial a_j} = \sum_k \delta_k \frac{\partial a_k}{\partial a_j}.$$

Como

$$a_k = \sum_r w_{kr} z_r, \quad z_j = h(a_j),$$

solo el término $r = j$ depende de a_j , y por tanto

$$\frac{\partial a_k}{\partial a_j} = w_{kj} \frac{\partial z_j}{\partial a_j} = w_{kj} h'(a_j).$$

Sustituyendo,

$$\delta_j = h'(a_j) \sum_k w_{kj} \delta_k.$$

Idea

La sensibilidad en una unidad oculta es la derivada local de su activación multiplicada por una suma ponderada de sensibilidades que vienen de la capa siguiente.

Fuente: Bishop, Ecuaciones (5.55)–(5.56).

1. Aplicar la entrada del dato y hacer **forward pass** para obtener todas las activaciones.
2. Calcular δ_k en las unidades de salida.
3. Propagar las δ hacia atrás en las unidades ocultas.
4. Evaluar las derivadas usando

$$\frac{\partial E_n}{\partial w_{ji}} = \delta_j z_i.$$

5. Si se usa batch, sumar sobre todos los datos:

$$\frac{\partial E}{\partial w_{ji}} = \sum_n \frac{\partial E_n}{\partial w_{ji}}.$$

Fuente: Bishop, resumen al final de la Sección 5.3.1.

Ejemplo: red de dos capas con $\tanh$ y salida lineal

Consideremos una red con:

- ▶ una sola capa oculta de tamaño M ;
- ▶ activaciones ocultas $\tanh$;
- ▶ salida lineal;
- ▶ error cuadrático para cada dato.

Las ecuaciones de forward son

$$a_j = \sum_{i=0}^D w_{ji}^{(1)} x_i, \quad z_j = \tanh(a_j), \quad y_k = \sum_{j=0}^M w_{kj}^{(2)} z_j.$$

Notación

El superíndice indica la capa de pesos: (1) para pesos entrada–oculta y (2) para pesos oculta–salida.

Fuente: Bishop, Sección 5.3.2.

En este ejemplo usamos dos matrices de pesos:

$$W^{(1)} = [w_{ji}^{(1)}]_{j=1,\dots,M}^{i=0,\dots,D}, \quad W^{(2)} = [w_{kj}^{(2)}]_{k=1,\dots,K}^{j=0,\dots,M}.$$

Capa 1

$$W^{(1)} \in \mathbb{R}^{M \times (D+1)}$$

Cada fila corresponde a una unidad oculta j .

Capa 2

$$W^{(2)} \in \mathbb{R}^{K \times (M+1)}$$

Cada fila corresponde a una salida k .

Sesgos

La columna $i = 0$ de $W^{(1)}$ y la columna $j = 0$ de $W^{(2)}$ representan sesgos, porque $x_0 = z_0 = 1$.

$$\mathbf{x} = \begin{bmatrix} x_0 \\ x_1 \\ \vdots \\ x_D \end{bmatrix}, \quad W^{(1)} = \begin{bmatrix} w_{10}^{(1)} & w_{11}^{(1)} & \cdots & w_{1D}^{(1)} \\ w_{20}^{(1)} & w_{21}^{(1)} & \cdots & w_{2D}^{(1)} \\ \vdots & \vdots & \ddots & \vdots \\ w_{M0}^{(1)} & w_{M1}^{(1)} & \cdots & w_{MD}^{(1)} \end{bmatrix} \quad \mathbf{z} = \begin{bmatrix} z_0 \\ z_1 \\ \vdots \\ z_M \end{bmatrix}, \quad W^{(2)} = \begin{bmatrix} w_{10}^{(2)} & w_{11}^{(2)} & \cdots & w_{1M}^{(2)} \\ w_{20}^{(2)} & w_{21}^{(2)} & \cdots & w_{2M}^{(2)} \\ \vdots & \vdots & \ddots & \vdots \\ w_{K0}^{(2)} & w_{K1}^{(2)} & \cdots & w_{KM}^{(2)} \end{bmatrix}$$

Idea

La notación matricial compacta solo empaqueta muchas ecuaciones escalares del mismo tipo.

Matricial

$$\mathbf{a}^{(1)} = W^{(1)}\mathbf{x}$$

$$\mathbf{z}_{1:M} = \tanh(\mathbf{a}^{(1)})$$

$$\mathbf{y} = W^{(2)}\mathbf{z}$$

Elemento a elemento

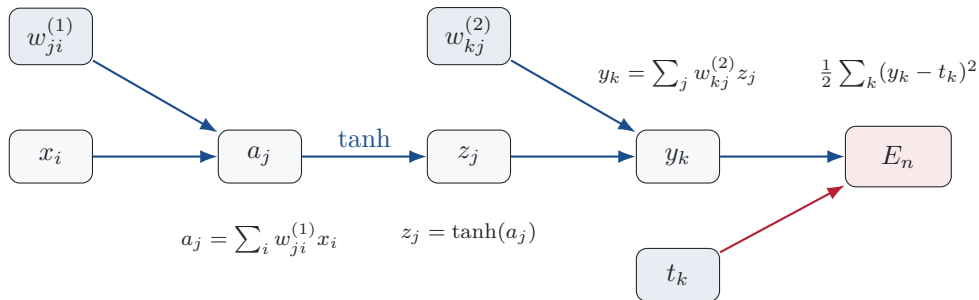
$$a_j = \sum_{i=0}^D w_{ji}^{(1)} x_i$$

$$z_j = \tanh(a_j)$$

$$y_k = \sum_{j=0}^M w_{kj}^{(2)} z_j$$

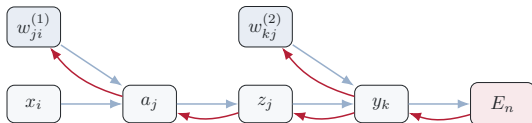
Cuidado

$\mathbf{z}_{1:M}$ excluye el sesgo $z_0 = 1$. En cambio, $\mathbf{z}$ completa incluye z_0 para poder escribir $\mathbf{y} = W^{(2)}\mathbf{z}$.



Idea

El grafo muestra las dependencias del forward pass. Backpropagation recorre estas dependencias en sentido inverso aplicando la regla de la cadena.



Piezas

$$\delta_k = \frac{\partial E_n}{\partial a_k} = y_k - t_k$$

$$\delta_j = (1 - z_j^2) \sum_k w_{kj}^{(2)} \delta_k$$

$$\frac{\partial E_n}{\partial w_{kj}^{(2)}} = \delta_k z_j, \quad \frac{\partial E_n}{\partial w_{ji}^{(1)}} = \delta_j x_i.$$

Idea

Las flechas rojas indican el flujo de derivadas. Primero se calculan las sensibilidades δ y después las derivadas respecto a los pesos.

Fuente: Inspirado en la lectura visual de grafos computacionales de Olah.

Para el error cuadrático,

$$E_n = \frac{1}{2} \sum_{k=1}^K (y_k - t_k)^2.$$

Entonces,

$$\delta_k = y_k - t_k.$$

Y para las unidades ocultas con $\tanh$,

$$\delta_j = (1 - z_j^2) \sum_{k=1}^K w_{kj}^{(2)} \delta_k,$$

porque

$$\frac{d}{da} \tanh(a) = 1 - \tanh^2(a).$$

Fuente: Bishop, Ecuaciones (5.61)–(5.66).

Las derivadas respecto a los pesos de la primera y segunda capa quedan:

$$\frac{\partial E_n}{\partial w_{ji}^{(1)}} = \delta_j^{(1)} x_i, \quad \frac{\partial E_n}{\partial w_{kj}^{(2)}} = \delta_k^{(2)} z_j.$$

Misma regla, distinta entrada

En ambas capas usamos

$$\frac{\partial E_n}{\partial w} = \text{sensibilidad posterior} \times \text{activación anterior}.$$

Para la primera capa, la activación anterior es el dato: $z_i \equiv x_i$. Para la segunda capa, la activación anterior es la unidad oculta: $z_j = \tanh(a_j)$.

Fuente: Bishop, Ecuación (5.67).

La forma matricial aparece porque las derivadas por elemento forman matrices completas.

Primera capa

$$(\nabla_{W^{(1)}} E_n)_{ji} = \frac{\partial E_n}{\partial w_{ji}^{(1)}} = \delta_j^{(1)} x_i$$

$$\boxed{\nabla_{W^{(1)}} E_n = \delta^{(1)} \mathbf{x}^T}$$

Segunda capa

$$(\nabla_{W^{(2)}} E_n)_{kj} = \frac{\partial E_n}{\partial w_{kj}^{(2)}} = \delta_k^{(2)} z_j$$

$$\boxed{\nabla_{W^{(2)}} E_n = \delta^{(2)} \mathbf{z}^T}$$

Lectura

Cada entrada de la matriz gradiente corresponde a un peso específico de la red.

Para un solo dato, escribiendo

$$\mathbf{x} = (x_0, x_1, \dots, x_D)^\top, \quad \mathbf{z} = (z_0, z_1, \dots, z_M)^\top,$$

con $x_0 = z_0 = 1$, primero calculamos las sensibilidades:

$$\boldsymbol{\delta}^{(2)} = \mathbf{y} - \mathbf{t},$$

$$\boldsymbol{\delta}^{(1)} = (1 - \mathbf{z}_{1:M}^2) \odot \left((W_{:,1:M}^{(2)})^\top \boldsymbol{\delta}^{(2)} \right).$$

Por qué se usa $W_{:,1:M}^{(2)}$

La columna 0 de $W^{(2)}$ corresponde al sesgo $z_0 = 1$. No se propaga hacia unidades ocultas porque z_0 no depende de ninguna activación previa.

Definición

El símbolo $\odot$ representa el **producto elemento a elemento**, también llamado **producto de Hadamard**. No es producto matricial.

Si $\mathbf{u}, \mathbf{v} \in \mathbb{R}^M$, entonces

$$\mathbf{u} \odot \mathbf{v} = \begin{bmatrix} u_1 \\ u_2 \\ \vdots \\ u_M \end{bmatrix} \odot \begin{bmatrix} v_1 \\ v_2 \\ \vdots \\ v_M \end{bmatrix} = \begin{bmatrix} u_1 v_1 \\ u_2 v_2 \\ \vdots \\ u_M v_M \end{bmatrix}.$$

Advertencia

Para que $\mathbf{u} \odot \mathbf{v}$ esté definido, ambos vectores deben tener la misma dimensión.

En la fórmula

$$\delta^{(1)} = (1 - \mathbf{z}_{1:M}^2) \odot \left((W_{:,1:M}^{(2)})^\top \delta^{(2)} \right),$$

los dos factores son vectores en $\mathbb{R}^M$:

$$1 - \mathbf{z}_{1:M}^2 = \begin{bmatrix} 1 - z_1^2 \\ \vdots \\ 1 - z_M^2 \end{bmatrix}, \quad (W_{:,1:M}^{(2)})^\top \delta^{(2)} = \begin{bmatrix} \sum_{k=1}^K w_{k1}^{(2)} \delta_k^{(2)} \\ \vdots \\ \sum_{k=1}^K w_{kM}^{(2)} \delta_k^{(2)} \end{bmatrix}.$$

Por lo tanto, para cada unidad oculta j :

$$\delta_j^{(1)} = (1 - z_j^2) \sum_{k=1}^K w_{kj}^{(2)} \delta_k^{(2)}.$$

Con las sensibilidades calculadas, el gradiente respecto a cada matriz de pesos es

$$\boxed{\nabla_{W^{(1)}} E_n = \boldsymbol{\delta}^{(1)} \mathbf{x}^\top}, \quad \boxed{\nabla_{W^{(2)}} E_n = \boldsymbol{\delta}^{(2)} \mathbf{z}^\top}.$$

Es decir, explícitamente:

$$\nabla_{W^{(1)}} E_n = \begin{bmatrix} \delta_1^{(1)} x_0 & \delta_1^{(1)} x_1 & \cdots & \delta_1^{(1)} x_D \\ \delta_2^{(1)} x_0 & \delta_2^{(1)} x_1 & \cdots & \delta_2^{(1)} x_D \\ \vdots & \vdots & \ddots & \vdots \\ \delta_M^{(1)} x_0 & \delta_M^{(1)} x_1 & \cdots & \delta_M^{(1)} x_D \end{bmatrix},$$

$$\nabla_{W^{(2)}} E_n = \begin{bmatrix} \delta_1^{(2)} z_0 & \delta_1^{(2)} z_1 & \cdots & \delta_1^{(2)} z_M \\ \delta_2^{(2)} z_0 & \delta_2^{(2)} z_1 & \cdots & \delta_2^{(2)} z_M \\ \vdots & \vdots & \ddots & \vdots \\ \delta_K^{(2)} z_0 & \delta_K^{(2)} z_1 & \cdots & \delta_K^{(2)} z_M \end{bmatrix}.$$

Si agrupamos todos los parámetros de la red en un solo vector

$$\mathbf{w} = \begin{bmatrix} \text{vec}(W^{(1)}) \\ \text{vec}(W^{(2)}) \end{bmatrix},$$

entonces el gradiente completo para el dato n es

$$\nabla_{\mathbf{w}} E_n = \begin{bmatrix} \text{vec}(\boldsymbol{\delta}^{(1)} \mathbf{x}^T) \\ \text{vec}(\boldsymbol{\delta}^{(2)} \mathbf{z}^T) \end{bmatrix}.$$

Lectura

La notación vectorizada no cambia el cálculo: solo apila, uno debajo de otro, todos los gradientes de $W^{(1)}$ y $W^{(2)}$.

Para evitar confundir L con el número de datos, aquí usaremos L para denotar el número de capas de pesos.

Definimos

$$\mathbf{z}^{(0)} = \mathbf{x},$$

y para cada capa $\ell = 1, \dots, L$:

$$\mathbf{a}^{(\ell)} = W^{(\ell)} \mathbf{z}^{(\ell-1)}, \quad \mathbf{z}^{(\ell)} = h^{(\ell)}(\mathbf{a}^{(\ell)}).$$

La salida final puede escribirse como

$$\mathbf{y} = \mathbf{z}^{(L)}.$$

Lectura

El superíndice (ℓ) indica la capa. La capa ℓ toma como entrada las activaciones de la capa anterior, $\mathbf{z}^{(\ell-1)}$.

Primero se calcula la sensibilidad de la capa de salida:

$$\delta^{(L)} = \frac{\partial E_n}{\partial \mathbf{a}^{(L)}}.$$

Después se propaga hacia atrás, para $\ell = L - 1, L - 2, \dots, 1$:

$$\delta^{(\ell)} = \left(h^{(\ell)'}(\mathbf{a}^{(\ell)}) \right) \odot \left((W^{(\ell+1)})^\top \delta^{(\ell+1)} \right).$$

Por elemento, para la unidad j de la capa ℓ :

$$\delta_j^{(\ell)} = h^{(\ell)'}(a_j^{(\ell)}) \sum_r w_{rj}^{(\ell+1)} \delta_r^{(\ell+1)}.$$

Idea

Es la misma regla de la cadena del caso de dos capas, aplicada repetidamente desde la salida hacia la entrada.

Una vez calculadas las sensibilidades $\delta^{(\ell)}$, el gradiente respecto a cada matriz de pesos es

$$\nabla_{W^{(\ell)}} E_n = \delta^{(\ell)} \left(\mathbf{z}^{(\ell-1)} \right)^T, \quad \ell = 1, \dots, L.$$

Equivalente por elemento:

$$\frac{\partial E_n}{\partial w_{ji}^{(\ell)}} = \delta_j^{(\ell)} z_i^{(\ell-1)}.$$

Si agrupamos todos los parámetros en un solo vector,

$$\mathbf{w} = \begin{bmatrix} \text{vec}(W^{(1)}) \\ \text{vec}(W^{(2)}) \\ \vdots \\ \text{vec}(W^{(L)}) \end{bmatrix}, \quad \nabla_{\mathbf{w}} E_n = \begin{bmatrix} \text{vec}(\nabla_{W^{(1)}} E_n) \\ \text{vec}(\nabla_{W^{(2)}} E_n) \\ \vdots \\ \text{vec}(\nabla_{W^{(L)}} E_n) \end{bmatrix}.$$

Hay dos maneras equivalentes de tratar los sesgos.

Sesgo absorbido

Se añade una coordenada constante:

$$z_0^{(\ell-1)} = 1.$$

Entonces el sesgo queda como la columna 0 de $W^{(\ell)}$ y

$$\frac{\partial E_n}{\partial w_{j0}^{(\ell)}} = \delta_j^{(\ell)}.$$

Sesgo separado

Se escribe

$$\mathbf{a}^{(\ell)} = W^{(\ell)} \mathbf{z}^{(\ell-1)} + \mathbf{b}^{(\ell)}.$$

Entonces

$$\nabla_{\mathbf{b}^{(\ell)}} E_n = \boldsymbol{\delta}^{(\ell)}.$$

Idea

La regla general no cambia: cada parámetro recibe la sensibilidad de su unidad multiplicada por la entrada que lo alimenta.

Para todo el conjunto de entrenamiento,

$$E(\mathbf{w}) = \sum_{n=1}^N E_n(\mathbf{w}).$$

Por linealidad de la derivada,

$$\nabla_{W^{(1)}} E = \sum_{n=1}^N \nabla_{W^{(1)}} E_n, \quad \nabla_{W^{(2)}} E = \sum_{n=1}^N \nabla_{W^{(2)}} E_n.$$

Idea

Backpropagation calcula el gradiente por dato; un método batch suma esos gradientes, mientras que SGD usa uno o pocos datos para actualizar los pesos.

Backpropagation combina cuatro ideas simples:

1. la estructura composicional de la red;
2. la regla de la cadena;
3. cantidades locales fáciles de interpretar (a_j, z_j, δ_j) ;
4. reutilización eficiente de cálculos ya hechos en el forward pass.

Idea

Backpropagation es una organización inteligente de la regla de la cadena para evitar recomputaciones masivas.

¿Por qué backpropagation es computacionalmente eficiente?

- ▶ Un forward pass en una red con W pesos cuesta típicamente $O(W)$.
- ▶ Un backward pass cuesta también $O(W)$.
- ▶ Por tanto, una evaluación completa del gradiente cuesta $O(W)$ por dato.

Advertencia

El gradiente completo se obtiene con un costo del mismo orden que evaluar la red, en lugar de requerir una perturbación separada por cada peso.

Fuente: Bishop, Sección 5.3.3.

La misma lógica puede extenderse a otras derivadas. En particular, interesa el **Jacobiano de la salida respecto a la entrada**:

$$J_{ki} = \frac{\partial y_k}{\partial x_i}.$$

- ▶ Mide la sensibilidad local de la salida y_k frente a cambios pequeños en la entrada x_i .
- ▶ Sus valores dependen del punto de entrada actual, porque la red es no lineal.

Idea

El Jacobiano describe cómo se *deforma localmente* el espacio de entrada bajo el mapa aprendido por la red.

1. Sensibilidad local y análisis de robustez.
2. Propagación de pequeñas perturbaciones o incertidumbres en la entrada.
3. Acoplamiento de módulos diferenciables dentro de sistemas más grandes.

Fuente: Bishop, discusión alrededor de la Figura 5.8.

Si Δx_i es pequeña, entonces la perturbación en la salida satisface aproximadamente

$$\Delta y_k \approx \sum_i \frac{\partial y_k}{\partial x_i} \Delta x_i.$$

Idea

La red se comporta localmente como una transformación lineal dada por su Jacobiano en el punto de entrada considerado.

Fuente: Bishop, Ecuación (5.72).

Partimos de

$$J_{ki} = \frac{\partial y_k}{\partial x_i}.$$

Aplicando regla de la cadena,

$$\frac{\partial y_k}{\partial x_i} = \sum_j \frac{\partial y_k}{\partial a_j} \frac{\partial a_j}{\partial x_i}.$$

Como

$$a_j = \sum_i w_{ji} x_i,$$

obtenemos

$$J_{ki} = \sum_j w_{ji} \frac{\partial y_k}{\partial a_j}.$$

Fuente: Bishop, Ecuación (5.73).

Las cantidades

$$\frac{\partial y_k}{\partial a_j}$$

pueden calcularse recursivamente mediante

$$\frac{\partial y_k}{\partial a_j} = h'(a_j) \sum_l w_{lj} \frac{\partial y_k}{\partial a_l}.$$

Idea

La forma de la recursión es análoga a la ecuación de backpropagation para las sensibilidades δ_j .

Fuente: Bishop, Ecuación (5.74).

La recursión del Jacobiano se inicia en la capa de salida, donde las derivadas dependen de la función de activación final.

- Si las salidas son sigmoides individuales:

$$\frac{\partial y_k}{\partial a_j} = \delta_{kj} \sigma'(a_j).$$

- Si la salida es softmax:

$$\frac{\partial y_k}{\partial a_j} = \delta_{kj} y_k - y_k y_j.$$

Fuente: Bishop, Ecuaciones (5.75)–(5.76).

Objeto	Símbolo	Qué mide
Gradiente respecto a pesos	$\partial E / \partial \mathbf{w}$	cómo cambia el error al variar parámetros
Jacobiano respecto a entradas	$\partial \mathbf{y} / \partial \mathbf{x}$	cómo cambia la salida al variar la entrada
Hessiano respecto a pesos	$\partial^2 E / \partial \mathbf{w} \partial \mathbf{w}$	curvatura local del error en el espacio de pesos

- ▶ Entrenar una red es un problema de optimización no convexa sobre el espacio de parámetros.
- ▶ El gradiente es información local crucial para hacer viable el entrenamiento.
- ▶ Backpropagation no es el entrenamiento completo: es un algoritmo eficiente para calcular derivadas.
- ▶ La misma lógica se extiende al Jacobiano de la red respecto a la entrada.